TECHNICAL CODE LEVEL

Code & Unix Deep-Dive

Algorithms, Subprocesses & Kernels

1. Kahn’s Algorithm in src/engine.py

The scheduler implements Kahn’s topological sort algorithm to order jobs and detect circular dependencies before spawning processes:

```
# Kahn’s Algorithm in src/engine.py
in_degree = {jid: 0 for jid in job_map}
adj_list = {jid: [] for jid in job_map}
for jid, job in job_map.items():
  for up in job.get(’depends_on’, []):
    adj_list[up].append(jid)
    in_degree[jid] += 1
queue = [j for j, d in in_degree.items()
if d == 0]
while queue:
  curr = queue.pop(0)
  ordered.append(curr)
  for n in adj_list[curr]:
    in_degree[n] -= 1
    if in_degree[n] == 0:
      queue.append(n)
if len(ordered) != len(job_map):
  raise ValueError(’Cyclic dependency!’)
```

2. True Wall-Clock SLA Calculation

SLA measurement must not rely on CPU execution time; it must measure real wall-clock latency including retry delays:

```
# Measuring Wall-Clock Duration
job_start_dt = datetime.now()
while attempt <= max_attempts:
  exit_code, out, err, dur = execute()
  if exit_code != 0 and attempt < max:
    time.sleep(retry_delay_sec)
    attempt += 1
job_end_dt = datetime.now()
total_sec = (job_end_dt -
job_start_dt).total_seconds()
sla_breached = 1 if total_sec > sla_max
else 0
```

3. SQLite WAL Mode Architecture

Database persistence in src/database.py uses SQLite Write-Ahead Logging (WAL) for high concurrency:

```
# WAL Mode & Pragmas in src/database.py
conn = sqlite3.connect(db_path)
conn.row_factory = sqlite3.Row
conn.execute('PRAGMA foreign_keys = ON;')
conn.execute('PRAGMA journal_mode = WAL;')
```

* **Why WAL?: Writers write to a separate -wal log** file. Readers never block writers, and writers never block readers.
* **Upsert Logic: Uses ON CONFLICT(run_id, job_id)** DO UPDATE to ensure idempotent status transitions.

4. Unix Housekeeping Commands

Exact bash commands in jobs/system_housekeeping.sh explained line-by-line:

Unix Command Reference

Disk Space:	df -h / | awk 'NR==2 {print $5, $4}'
Memory Usage:	free -m | awk '/^Mem:/ {print $2, $7}'
CPU Load Avg:	awk '{print $1}' /proc/loadavg
Zombie Check:	ps aux | awk '$8=="Z"' | wc -l
Spool Purge:	find spool_tmp/ -name '*.tmp' -delete
Daemon Check:	Verifies PIDs & Ports (7005)

ZOMBIE PROCESSES EXPLAINED:
A zombie process (state Z in ps aux) has terminated, but its parent has not read its exit status with wait(). Zombies consume zero CPU/RAM, but they consume PID slots. Kill -9 cannot kill a zombie! You must send SIGCHLD to the parent (kill -s SIGCHLD <ppid>) or restart the parent.